Semantic Data Management

Universitat Politècnica de Catalunya (UPC)

Property Graphs Lab

Henriette Skarveland Nystad Attila Istvan Kiri

March 2026

1 Part A — Graph Modelling and Loading

A.1 — Property Graph Model

This section covers the graph model designed to represent a research publication domain in Neo4j. The domain includes authors writing papers that are published either in conference/-workshop proceedings or in journals. Conferences and workshops are organised in editions, each held at a specific city and year. Journals publish papers in volumes, potentially multiple per year. Papers can cite other papers, can be tagged with keywords, and must have an abstract. Each paper has a single corresponding author. When submitted, papers are assigned a set of reviewers — typically three — who must not be authors of the paper being reviewed. The goal of this section is to design a visual and documented schema capturing all these concepts as nodes and edges.

Describe the design decisions behind your graph model. Explain which concepts you chose to represent as nodes and which as relationships, and justify the choice of properties assigned to each.

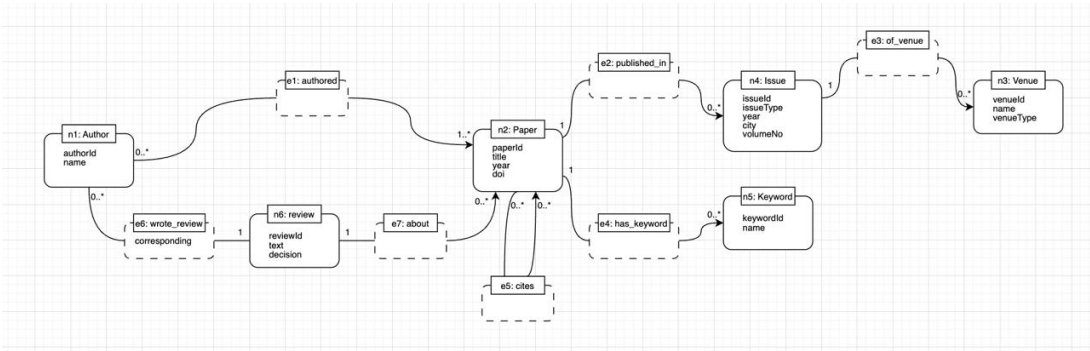

Figure 1: Property graph schema for research paper dataset.

Table 1: Node types and their key properties.

Label	**Properties**	**Description**
`Author`	`authorId`, `name`	A researcher who writes and reviews papers. Exists in
`Paper`	`paperId`, `title`, `year`, `doi`	A published academic work, either a conference paper or journal article.
`Review`	`reviewId`, `text`, `decision`	A peer-review of a paper, containing the reviewer's comments and acceptance decision (accept, reject, minor revision, or major revision).
`Issue`	`issueId`, `issueType`, `year`, `city`, `volumeNo`	Represents either a conference/-workshop edition (`issueType` = *proceedings*, with a host `city`) or a journal volume (`issueType` = *volume*, with a `volumeNo`). This single node type unifies the concepts of edition and volume.
`Venue`	`venueId`, `name`, `venueType`	A publication venue. `venueType` is one of *Conference*, *Workshop*, or *Journal*.
`Keyword`	`keywordId`, `name`	A topic keyword describing the subject area of a paper (e.g. *knowledge graph*, *RDF*, *graph neural network*).

Table 2: Edge types, their endpoints, and properties.

Edge	From	To	Description
`authored`	`Author`	`Paper`	Connects an author to a paper they have written. A paper must have at least one author (1..∗).
`wrote_review`	`Author`	`Review`	Connects a reviewer to the review they submitted. Carries the Boolean property `corresponding`, which is `true` for the single author designated as the corresponding reviewer and `false` for all others.
`about`	`Review`	`Paper`	Links a review to the paper it evaluates. Each review is about exactly one paper (1).
`published_in`	`Paper`	`Issue`	Records in which issue (proceedings or volume) a paper was published. Each paper is published in exactly one issue (1).
`of_venue`	`Issue`	`Venue`	Associates an issue with the venue to which it belongs (e.g. a proceedings edition belongs to a conference, a volume belongs to a journal).
`has_keyword`	`Paper`	`Keyword`	Tags a paper with one or more domain keywords (0..∗).
`cites`	`Paper`	`Paper`	Directed edge from a citing paper to a cited paper, representing a bibliographic reference (0..∗).

A.2 — Instantiating / Loading

This task covers the process of populating the graph designed in A.1 with real-world and synthetic data inside Neo4j.

Real Data

The data was obtained from Data was obtained from the **DBLP Computer Science Bibliography** (https://dblp.org) as suggested in the project description. DBLP makes its entire dataset available as a single XML dump, which was converted to semicolon-delimited CSV files using the `dblp-to-csv` github repository, as suggested in the project description.

Several node and edges required by the graph model (Section A.1) are not exported directly by `dblp-to-csv` but can be *derived* from the data already present in the files.

The following raw DBLP files were used as the primary source of real data:

- `output_author and authored by exists as author.csv and authored.csv`.
- `output_paper exists as article.csv and as inproceedings.csv`.
- `output_proceedings.csv` — proceedings records (booktitle, year, address).
- `output_journal.csv` — journal metadata (name, volume, year). (paper key → author name).
- `output_Citations exists as cite.csv` / `output_cite_has_citation.csv` — citation edges between papers.

- `output_journal_published_in.csv` — paper-to-journal membership edges.

Creating Synthetic Data

Some data are not present in DBLP and were therefore generated synthetically using Python's

The following synthetic data was created for the project:

- `output_Review, wrote review and about has been created as review.csv, wrote review.csv, about.csv`
- `output_Keywords has been created as keyword.csv` — journal articles (title, year, journal, volume).

Loading into neo4j

After all Csvs have been pre-processed we used Neo4j's `LOAD CSV` function to load in all the files cleaned into the database and this was the view of the model.

A.3 — Graph Evolution

This section demonstrates the flexibility of graph databases to accommodate schema and data changes without requiring a full redesign. Two extensions are introduced to the model from A.1. First, the review concept is enriched: rather than simply linking a paper to its reviewers, the graph now stores the full review as a separate entity, including its textual content, suggested decision, and the policy that a paper is accepted if a majority of reviewers support acceptance. Second, author affiliations are introduced — each author can now be linked to an organisation (university or company) via an `AFFILIATED_WITH` relationship. The schema update was applied to the live Neo4j graph using Cypher queries implemented in `FormatUpdateCSV.py` and `UploadUpdateCSV.py`, without requiring a full reload of the database.

A/A.3/Schema.png

Figure 2: Updated property graph schema after A.3 modifications.

Explain what was added or changed compared to A.1 and how you loaded the new data using `FormatUpdateCSV.py` and `UploadUpdateCSV.py`.

2 Part B — Queries

This section presents four Cypher queries that exploit the graph constructed in Part A. The goal is to write efficient queries that minimise disk accesses and intermediate result sizes. For each query, the question being answered and the key design choices in the Cypher logic are described.

B.1 — Top 3 Most Cited Papers per Conference/Workshop

This query finds, for each conference or workshop, the three papers with the highest number of incoming `CITES` relationships. The approach groups papers by the venue they were published in, counts their citations, and uses `ORDER BY` with `COLLECT` and list slicing (or `LIMIT` within a subquery) to retain only the top 3 per venue. This query is useful for identifying landmark contributions within each community.

Describe query B1 here.

B.2 — Community Authors per Conference/Workshop

This query identifies, for each conference or workshop, the set of authors who have published there in at least four different editions. It traverses `AUTHORED` and `PUBLISHED_IN` relationships, groups by author and venue, and filters on the count of distinct editions. These recurring contributors form the core community of a venue and are relevant for reviewer selection in Part C.

Describe query B2 here.

B.3 — Impact Factor of Journals

This query computes the impact factor for each journal in the graph. The impact factor is approximated as the average number of citations received per paper published in the journal over a defined window (typically the two preceding years). The query aggregates citation counts for papers published in each journal volume and computes the ratio, surfacing the most impactful journals in the dataset.

Describe query B3 here.

2.1 B.4 — H-Index of Authors

This query computes the h-index for every author in the graph. An author has h-index h if they have at least h papers each receiving at least h citations. The query collects the citation counts of all papers per author, sorts them in descending order, and finds the largest h satisfying the definition. This metric is widely used to assess individual researcher impact.

Describe query B4 here.

3 Part C — Recommender

This section implements a multi-step reviewer recommender system for the database research community. Each step is implemented as a single Cypher query that both retrieves and *asserts* (writes) the inferred knowledge back into the graph, so that subsequent steps can build on it. The process identifies the database community, finds its associated venues, selects the top papers, and ultimately labels suitable reviewers and gurus.

C.1 — Define the Database Community

The database community is defined by a fixed set of keywords: *data management*, *indexing*, *data modeling*, *big data*, *data processing*, *data storage*, and *data querying*. This step creates a `Community` node (or equivalent label) connected to these keywords via `HAS_TOPIC` relationships, asserting the community definition into the graph for reuse in subsequent steps.

Describe the query/update C1 here.

C.2 — Identify Database Venues

This step finds conferences, workshops, and journals that are specific to the database community. A venue is considered related to the community if at least 90% of its published papers contain at least one of the community's keywords. The query computes this ratio per venue and asserts a `RELATED_TO` relationship between matching venues and the community node created in C.1.

Describe the query/update C2 here.

C.3 — Select Top-100 Papers of the Database Community

From the venues identified in C.2, this step selects the 100 papers with the highest number of citations coming from other papers in the database community. The query counts only intra-community citations (via `CITES` edges where the citing paper also belongs to a database venue), ranks papers accordingly, and asserts a `TOP_PAPER` label or relationship on the resulting 100 papers.

Describe the query/update C3 here.

C.4 — Label Potential Reviewers and Gurus

Any author of at least one top-100 paper is marked as a potential reviewer for database papers. Authors who have written at least two top-100 papers are additionally labelled as *gurus* — reputable researchers considered suitable to review for top-tier venues. This step asserts `POTENTIAL_REVIEWER` and `GURU` labels (or equivalent relationships) onto qualifying author nodes in the graph.

Describe the query/update C4 here.

4 Part D — Graph Algorithms

This section applies two algorithms from the Neo4j Graph Data Science (GDS) Library to the research publication graph. The library must be installed as a plugin in the Neo4j instance. Each algorithm is contextualised to the publication domain: the choice of node label, relationship type, and parameters must be semantically meaningful, and the results are interpreted from a domain perspective. The following algorithm families were studied: centrality (PageRank, Betweenness, Closeness), community detection (Triangle Counting, Louvain, Connected Components), similarity (Node Similarity), and path finding (Dijkstra's Shortest Path).

D.1 — [Algorithm Name]

Explain which algorithm was used, why it is suitable for this question, and interpret the results obtained.

D.2 — [Algorithm Name]

Explain which algorithm was used, why it is suitable for this question, and interpret the results obtained.